

Project Explanation — Fake News Detection System

What Is This Project?

This is a **Fake News Detection System** — a full-stack web application that takes any piece of news content (text, a website URL, or an image) and tells you whether it is **REAL** or **FAKE**, along with a confidence score and a detailed explanation of why.

The system does not rely on a single algorithm. It uses multiple AI layers working together to reach the most accurate conclusion possible.

Why Was It Built?

Misinformation spreads rapidly through social media and messaging platforms. A single false news story can influence elections, cause public panic, or damage reputations before it is ever fact-checked.

Traditional approaches to detecting fake news fall short because:

- A single ML model can recognize writing style but cannot check facts.
- Rule-based systems catch known patterns but miss new forms of misinformation.
- LLMs can reason but hallucinate or make errors alone.

This system combines all three approaches so that their individual weaknesses cancel each other out.

How Does It Work? (Simple Explanation)

When you submit a piece of news, the system runs it through three layers:

Layer 1 — Machine Learning (DistilBERT)

A custom-trained AI model reads the text and classifies it statistically based on patterns it learned from **94,439 real and fake news articles** across multiple datasets.

It produces:

- A verdict: REAL or FAKE
- A confidence score (e.g., 66.5%)

Layer 2 — Large Language Model (Gemini / OpenAI / Ollama)

A powerful language model reads the same content and reasons about it — like a detective evaluating a claim. It looks for:

- Sensational or fear-based language
- Missing source citations
- Implausible claims
- Logical inconsistencies

It also produces:

- A verdict: Real or Fake
- A confidence score (e.g., 100%)
- A reasoning explanation
- Key signals detected

Layer 3 — Fusion Engine

The fusion engine takes both results and combines them using a weighted voting system. The more confident each layer is, the more influence it has on the final decision.

Example from a real run:

```
ML model  → FAKE at 66.5%  (medium confidence, weight 0.55)
LLM       → FAKE at 100%   (high confidence,   weight 0.75)

Both agree → Final: FAKE at 88.4% → "Definitely Fake"
```

What Makes This System Different?

Feature	Why It Matters
Two AI layers (ML + LLM)	One catches stylistic patterns, the other reasons about meaning
Fusion scoring	Neither layer decides alone — evidence is weighted and combined
Multilingual support	Non-English news is auto-detected and translated before analysis
Three input types	Text, URLs, and images are all supported
Per-user history	Every user's predictions are stored privately under their account
Multiple LLM providers	Gemini → OpenAI → Ollama fallback means the system keeps working even if one provider is down

The Machine Learning Model

The core ML model is based on **DistilBERT** — a lightweight, fast version of Google's BERT transformer. The multilingual variant was chosen because:

- It supports 104 languages out of the box.
- It is faster and smaller than BERT while retaining ~97% of its accuracy.
- It can be fine-tuned on custom datasets.

The model was fine-tuned from scratch on a merged dataset combining:

Dataset	What It Contains
---------	------------------

Dataset	What It Contains
ISOT	Political and world news (English)
WELFake	Diverse news from 4 major datasets
FakeNewsNet	Celebrity gossip and political fact-checks
LIAR	Short political statements with truth ratings

After cleaning, merging, and deduplication, **94,439 articles** were used. The data was split 70/15/15 into train, validation, and test sets.

Results after training for 3 epochs on an NVIDIA RTX 2050:

Metric	Score
Accuracy	91.5%
Precision	91.1%
Recall	87.9%
F1 Score	0.895

The LLM Layer

The LLM verification layer adds **reasoning** on top of the statistical ML result.

The system sends the news content to a language model with a structured prompt asking it to return a JSON verdict, confidence score, reasoning paragraph, and key detected signals.

Provider priorities (auto mode):

- 1. **Gemini** — if `GEMINI_API_KEY` is set
- 2. **OpenAI** — if `OPENAI_API_KEY` is set
- 3. **Ollama** — local fallback (no API key required, runs on your machine)

If the primary provider fails for any reason, it automatically falls back to Ollama.

Handling Different Input Types

Plain Text

The user types or pastes a news article. The system detects the language, translates to English if needed, and runs the full pipeline.

URL

The user pastes a news link. The system:

- 1. Scrapes the article using `newspaper3k` (with a BeautifulSoup fallback)
- 2. Extracts the title, author, date, and body text

- 3. Runs the full pipeline on the extracted content

Image

The user uploads a screenshot of a news story. The system:

- 1. Runs Tesseract OCR to extract text from the image
- 2. Runs the full pipeline on the extracted text
- 3. Optionally uploads the image to Cloudinary for storage

How Results Are Explained

Every prediction includes:

- **Verdict:** FAKE or REAL
- **Confidence:** 0–100%
- **Risk Level:** Low / Medium / High
- **Assessment:** Definitely Fake / Likely Fake / Suspicious / Likely Real / Definitely Real
- **ML breakdown:** What the ML model predicted and at what confidence
- **LLM breakdown:** What the LLM said, its reasoning, and the signals it detected
- **Fusion analysis:** Fake score vs Real score and how many evidence items contributed

The User Account System

Every user has their own account. All predictions, history, analytics, and profile settings are stored privately in MongoDB Atlas under their email address.

- **Signup / Login:** Register with name, email, and password
- **Profile:** Update name, phone, organization, role
- **Password change:** Stored securely as a bcrypt hash in MongoDB
- **History:** View all past predictions with timestamps
- **Analytics:** Charts showing prediction trends over time
- **Delete account:** Permanently removes all data

Tech Stack

Backend

Technology	Purpose
Python 3.10	Core language
Flask	REST API framework
PyTorch + Transformers	ML model inference
MongoDB Atlas	Database
bcrypt	Password hashing

Technology	Purpose
Tesseract OCR	Image text extraction
deep-translator	Language translation
Cloudinary	Image storage
newspaper3k + BeautifulSoup	URL article scraping

Frontend

Technology	Purpose
React + TypeScript	UI framework
Vite	Build tool
Tailwind CSS	Styling
React Router	Page navigation

AI/ML

Technology	Purpose
distilbert-base-multilingual-cased	Base pre-trained model
HuggingFace Trainer API	Fine-tuning framework
Gemini / OpenAI / Ollama	LLM reasoning layer

Known Limitations

1. **ML detects style, not truth** — A well-written fake article may fool the ML layer. The LLM partially compensates for this.
2. **Long articles are truncated** — The ML model has a 512 token limit. Very long articles lose tail content.
3. **LLM requires an API key or Ollama** — Without any LLM configured, only the ML layer runs.
4. **URL scraping can fail** — Heavily JavaScript-rendered pages may not extract clean article text.
5. **OCR quality affects image results** — Blurry or low-resolution screenshots will produce noisy text.
6. **Dataset bias** — Training data is mostly English and US-focused news. Performance on regional Indian language news may vary.

What Can Be Improved

- Activate the knowledge/rule layer inside the main predict pipeline for stronger factual checks.
- Add source credibility check automatically for URL predictions.
- Integrate Google Fact Check API for real-time ground-truth comparison.
- Replace local browser-based auth with a full backend authentication system.
- Export the model to ONNX for faster CPU inference.
- Add SHAP/LIME explainability to highlight the exact words that drove the ML prediction.

Summary

The Fake News Detection System is a 3-layer hybrid AI pipeline that combines:

```
DistilBERT (statistical pattern recognition)
+
LLM (contextual reasoning)
+
Fusion Engine (evidence weighting)
=
Reliable fake news verdict with confidence and explanation
```

It supports text, URL, and image inputs, stores results per user in MongoDB, handles multiple languages, and falls back gracefully when external services are unavailable.